

RAM vs. Word-RAM, and the Church–Turing Thesis

CS1200: Introduction to Algorithms and their Limitations

Lecture 10

Fall 2026

Harvard SEAS

Recap: a Word-RAM program

A **Word-RAM program** is a RAM program with a **word length** w (fixed for the run) and a **memory size** S (starts at n ; MALLOC adds one cell). Values live in $\{0, \dots, 2^w - 1\}$; $+$ and $\times$ saturate at $2^w - 1$. $P[w](x)$ halts with an output, runs forever, or crashes.

Def. 8.2. P **solves** $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if for every $x \in \mathcal{I}$:

1. _____
2. _____
3. _____

Def. 8.3. $T_P(x) = \underline{\hspace{2cm}}$ (crashed runs count; ∞ if unbounded).

Check: three programs, one input

Consider a problem Π and an input x with $f(x) = \{(5)\}$: the only correct output on x is (5). Each cell below is the outcome of a program run at that w and its number of steps.

	$w = 2$	$w = 3$	$w \geq 4$
P	crash, 3	output (5), 12	output (5), 12
Q	crash, 3	output (4), 12	output (5), 12
R	runs forever	output (5), 12	output (5), 12

Which of these programs do *not* solve Π ? Why? What is $T(x)$?

P : _____ $T_P(x) =$ _____

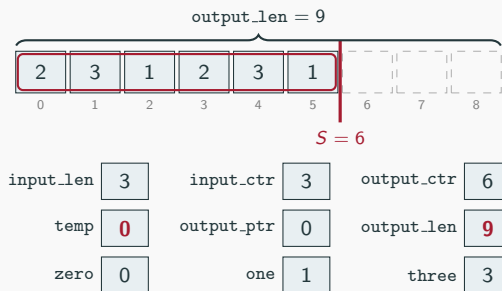
Q : _____ $T_Q(x) =$ _____

R : _____ $T_R(x) =$ _____

Duplicate

Input: an array x of length n . **Output:** the array of length $2n$: x followed by x . Here $x = (2, 3, 1)$, and w is large enough not to matter.

```
0 zero = 0;
1 one = 1;
2 three = 3;
3 input_ctr = 0;
4 output_ctr = input_len;
5   temp = input_len - input_ctr;
6   IF temp == 0 GOTO 13;
7   temp = M[input_ctr];
8   MALLOC;
9   M[output_ctr] = temp;
10  input_ctr = input_ctr + one;
11  output_ctr = output_ctr + one;
12  IF zero == 0 GOTO 5;
13 output_ptr = 0;
14 output_len = three × input_len;
```



Line 5: $temp = 0$, so line 6 jumps to 13. Lines 13–14 ask for 9 cells from $M[0]$; only $S = 6$ exist, so the output is cut at $M[S - 1]$: $(2, 3, 1, 2, 3, 1)$.

In groups: Duplicate at every word length

```
0 zero = 0;
1 one = 1;
2 three = 3;
3 input_ctr = 0;
4 output_ctr = input_len;
5   temp = input_len - input_ctr;
6   IF temp == 0 GOTO 13;
7   temp = M[input_ctr];
8   MALLOC;
9   M[output_ctr] = temp;
10  input_ctr = input_ctr + one;
11  output_ctr = output_ctr + one;
12  IF zero == 0 GOTO 5;
13 output_ptr = 0;
14 output_len = three × input_len;
```

1. For which word lengths w does $P[w](x)$ crash? At which step, and why?

2. How many commands does $P[w](x)$ execute for $w \geq 3$? For $w = 2$?

3. $T_P(x)$ is the maximum of $T_{P[w]}(x)$ over all w . What is it? What is the worst case $T_P(n)$?

Proposition 8.4

Recall (Def. 8.1): $P[w](x)$ crashes only if (1) a constant in P is $\geq 2^w$, (2) an input entry is $\geq 2^w$, or (3) $S > 2^w$.

Prop. 8.4. If $T_P(x) < \infty$, then $P[w](x)$ does not crash for any $w \geq w_0$, where

$$w_0 = \left\lceil \log_2 \max \left\{ \underbrace{n + T_P(x)}_{\text{bounds } S \text{ (3)}}, \underbrace{x[0], \dots, x[n-1]}_{\text{input entries (2)}}, \underbrace{c_0, \dots, c_{k-1}}_{\text{constants in } P \text{ (1)}} \right\} + 1 \right\rceil.$$

Why does $n + T_P(x)$ bound S ? _____

Consequence: if $T_P(n) = \text{poly}(n)$ and inputs are at most $\text{poly}(n)$, then $w = \rule{1cm}{0.4pt}$ suffices.
(The constants are fixed: they don't grow with n .)

For a fixed machine, P is guaranteed to work on every input small enough that $w \geq w_0$. A 64-bit machine covers any realistic input.

Above w_0 , the **model** guarantees no crash; the **programmer** guarantees no wrong answer.

What does the Word-RAM model miss?

Word-RAM has been the dominant model in algorithms for over 50 years. Still:

Last lecture's desiderata pull against each other; the Word-RAM balances simplicity, robustness, and technological relevance.

Everything is an array of natural numbers

Inputs and outputs of RAM programs are arrays of natural numbers. How do we represent:

signed integers _____

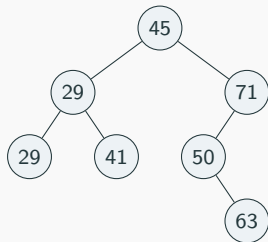
rationals _____

real numbers _____

strings _____

Floating point ($x \cdot 2^y$) approximates reals, and almost every operation rounds. This course sticks to integers and rationals.

A binary search tree as an array



Each vertex is a 4-tuple (K, V, P_L, P_R) : key, value, and the *addresses* of its left and right children. 0 means “no child”.

Why can we use 0 to mean “no child”?

K	V	P_L	P_R																											
45	0	4	8	29	0	12	16	71	0	20	0	29	0	0	0	41	0	0	0	50	0	0	24	63	0	0	0			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27			

Runtimes of Word-RAM simulations of RAM programs

Thm. 8.5. Fix an encoding of inputs as arrays. Let $T(x) \geq n + 2$, with every input entry of bitlength $O(\log T(x))$. These are equivalent (either both are true or both are false):

1. Π is solved by a Word-RAM program P with $T_P(x) = O(T(x))$.
2. Π is solved by a RAM program Q with $T_Q(x) = \underline{\hspace{2cm}}$ such that, on every x ,
 - (a) $\underline{\hspace{10cm}}$
 - (b) $\underline{\hspace{10cm}}$

Q: What can a RAM do in one step that a Word-RAM can't do cheaply?

Cor. 8.6. A problem is solvable by a Word-RAM program $\underline{\hspace{2cm}}$ it is solvable by a RAM program.*

*Without any efficiency guarantee: the Word-RAM program may be much slower.

Proof idea

Recall (Def. 8.2): P solves Π if, for every x : (1) on every w , $P[w](x)$ halts or crashes; (2) for all large enough w , it does not crash; (3) whenever it does not crash, its output is correct. Runtime is the max over all w , crashed runs included.

(2) $\Rightarrow$ (1): a Word-RAM program P imitates a non-cheating RAM program Q .

P does not know $T(x)$, so it guesses: $t = 1, 2, 4, 8, \dots$. For each guess t :

1. MALLOC about t cells: room for Q 's memory (uses (b)).
 - *Small machine*: it crashes here. Allowed by (2); it took $O(t)$ steps.
 - *Survives*: then $2^w \geq t$, so $w \geq \underline{\hspace{2cm}}$. Each of Q 's values fits in $O(1)$ words (uses (a)).
2. Simulate Q for t steps, $O(1)$ steps each. If Q finishes, output its answer: correct, so (3) holds. If not, double t .

Once $t \geq T(x)$ an attempt succeeds. Total: $1 + 2 + 4 + \dots \leq \underline{\hspace{2cm}}$.

(1) $\Rightarrow$ (2): a RAM program imitates a Word-RAM program You solve this for PS4.

Turing-equivalent models

Thm. 7.5. If a problem is solvable in one of these models, it is solvable in all of them:

- RAM programs
- $\%$ -extended RAM programs
- Word-RAM programs
- ...
- _____

Such models are called **Turing-equivalent**.

This is a **theorem**: each pair is linked by a simulation.

The Church–Turing Thesis

Two variants:

1. _____
2. _____

Why can't we prove it? _____

It has held up for 90 years — including quantum computers: anything a quantum algorithm can solve, a RAM program can solve, much more slowly.

The Extended Church–Turing Thesis

The Church–Turing Thesis is about *what* can be solved. The Extended Church–Turing Thesis is about *how fast*.

v1. _____

Possible challenges: _____

v2. _____

Models polynomially equivalent to the Word-RAM (such as Turing machines) are **strongly Turing-equivalent**.

Theorem, thesis, or false?

1. Every Python program can be simulated by a RAM program. _____
2. A problem is solvable by a Word-RAM program iff it is solvable by a RAM program.

3. Anything a physical device can compute, a RAM program can compute. _____
4. A RAM program running in time T can be simulated by a Word-RAM program in time $O(T)$.

5. Every deterministic, sequential physical computer can be simulated by a Word-RAM program with polynomial slowdown. _____
6. Quantum computers can solve problems that no RAM program can solve. _____

Takeaway

- The Word-RAM is the formal model under everything in this course.
- In practice: analyze the friendlier RAM model and check Theorem 8.5's conditions — no huge numbers, no huge addresses.
- Every algorithm so far passes that check, so their runtimes (InsertionSort n^2 , MergeSort $n \log n$, RadixSort, ...) are honest Word-RAM runtimes.
- From now on: high-level pseudocode. The models are the reference for what one step costs.